On the Pitfalls of Applying Architecture-agnostic ML Models to Learn the Utilization of Individual Tasks while Scheduling Mixed Criticality Tasksets on Multicore Platforms

Eric Fernandes Evaristo, José Luis Conradi Hoffmann and Antônio Augusto Fröhlich, *Member, IEEE,*
Software/Hardware Integration Lab, *Federal University of Santa Catarina*, Florianópolis, Brazil
{fernandes,hoffmann,guto}@lisha.ufsc.br

***Abstract*—We demonstrate inference accuracy restored to X%, schedulability maintained with Y% temporal overhead, yielding Z% energy savings over baseline models that fail under global PMU noise.**

***Index Terms*—Real-time embedded systems, Hardware Performance Counters, Machine Learning, Contention Inference, DVFS, Opaque Architecture.**

I. Introduction

Real-time Operating Systems (RTOS) are characterized by restrictive timing constraints and determinism, which make them suitable for the execution of critical applications. These systems are commonly employed in embedded scenarios, where other variables must also be considered, such as energy consumption [1]. When timing and energy consumption constraints are combined with the current widespread adoption of multicore systems, the algorithms responsible for scheduling and load balancing face a considerable challenge [2]–[4]. This scenario motivates the need for predictive algorithms capable of estimating the utilization of each task.

In recent years, several works have proposed the use of Performance Monitoring Counters (PMCs) in predictive algorithms, relying on hardware information to guide decisions. Most of these approaches rely on Machine Learning (ML) techniques, however, they also report difficulties and pitfalls that hinder their adoption. ML solutions usually introduce prohibitive overhead and fail to generalize well to new task-sets and platforms [1]. Moreover, the use of PMCs makes it difficult to design architecture-agnostic predictive algorithms. Although some works propose solutions to this issue, they remain limited by problems such as the divergence of PMC semantics across architectures, lack of counters in some platform implementations and also non-determinism and noise introduced by the Operating System (OS), applications or other external sources [5].

Considering this scenario, this paper addresses the main problems associated with the use of PMCs in ML algorithms, offering an analysis based on data collected from a RTOS running on RISC-V and ARM platforms. We also propose an architecture-agnostic solution that overcomes the identified problems and evaluate its performance using several real-time task-sets.

- An analysis of the main pitfalls of using PMCs in ML algorithms;
- An architecture-agnostic solution that addresses the identified problems with PMCs usage;
- Evaluation of the proposed solution on a real RISC-V platform using several task-sets.

II. Related Work

In our prior work, we explored the application of a PMC-fed lightweight Artificial Neural Network (ANN) for task utilization prediction in platforms with ARMv7, ARMv8 and RISC-V architectures [6], [7].

A. Predictive Energy Management in RTES

B. RISC-V Performance Monitoring

III. System Model

To evaluate the portability of the predictive energy optimization framework, we adopt a standard multicore real-time scheduling model, augmented with the specific microarchitectural constraints imposed by the target Commercial Off-The-Shelf (COTS) RISC-V platform.

We consider a multicore System-on-Chip (SoC) comprising a set of M identical hardware threads (harts), denoted as $\mathcal{M} = \{m_1, m_2, \ldots, m_M\}$. The system supports Dynamic voltage and frequency scaling (DVFS), providing a finite set of discrete operating frequencies $\mathcal{F} = \{f_{min}, \ldots, f_{max}\}$. Dynamic power dissipation scales proportionally to the voltage and frequency ($P \propto V^2 f$).

Crucially, the cores in $\mathcal{M}$ are not necessarily independently scalable. They are organized into frequency clusters (or voltage/frequency domains). Let $\mathcal{V} = \{v_1, \ldots, v_K\}$ be the set of clusters, where each cluster $v_k \subseteq \mathcal{M}$ shares a single clock and voltage supply. Consequently, a DVFS actuation applied to any core $m \in v_k$ simultaneously forces all cores within v_k to transition to the newly selected frequency $f \in \mathcal{F}$.

The workload consists of a taskset $\Gamma = \{\tau_1, \tau_2, \ldots, \tau_N\}$ of independent, periodic, or sporadic real-time tasks. At the

operating system level, tasks are scheduled using Partitioned Earliest Deadline First (PEDF). The global taskset Γ is statically partitioned into disjoint subsets Γ_m assigned to each core $m \in \mathcal{M}$. Each core maintains a local ready queue and schedules its assigned workload independently using the preemptive EDF policy.

Each task τ_i is formally defined by the tuple (C_i, D_i, T_i), where:

- C_i is the Worst-Case Execution Time (WCET) measured in strict isolation at the maximum frequency f_{max}.
- D_i is the relative deadline, representing the maximum allowable response time.
- T_i is the period or minimum inter-arrival time between consecutive job releases.

We assume constrained or implicit deadlines ($D_i \leq T_i$). Under dynamic run-time conditions, the actual execution time of a job belonging to task τ_i, denoted as C_{actual}, deviates significantly from C_i. This deviation is a nonlinear function of the assigned cluster frequency $f \in \mathcal{F}$ and the inter-core interference $I(t)$ generated by co-scheduled tasks contending for shared resources (e.g., memory buses and Last-Level Cache (LLC)). Because a frequency downscale affects all cores in v_k, predicting C_{actual} accurately for all co-located tasks is mandatory. The objective of the framework is to predict C_{actual} under lower frequencies to actuate DVFS safely at the cluster level, ensuring $C_{actual} \leq D_i$ for all tasks in v_k while minimizing energy consumption.

IV. Architecture-Agnostic Predictive Framework

Accurately modeling the impact of shared resource contention on task execution time requires abstracting highly complex, platform-specific microarchitectural behaviors. However, modern platforms expose hardware and operating system-level performance traces that serve as deterministic indicators of temporal degradation. Given that the primary optimization objective is energy minimization, the scheduler must precisely bound how aggressively DVFS can be applied without violating hard real-time constraints.

As established in the baseline framework [6], identifying these architectural bottlenecks without relying on vendor-specific heuristics requires a generalized, automated approach to performance event selection. Before run-time prediction occurs, the platform is profiled using a carefully tailored task-set to collect a comprehensive array of available hardware and operating system events. To isolate the most deterministic indicators of shared resource contention, a merged feature selection strategy is applied. This multi-tiered pipeline combines three distinct analytical classes: statistical correlation (Pearson), embedded regularization (Lasso Cross-Validation), and entropy-based filtering (Information Gain).

The resulting optimal feature vector forms the input for an ANN, which is continuously trained online to predict the temporal penalty of a proposed CPU frequency reduction. This prediction enables a cluster-wide voting mechanism, allowing each core to verify if its assigned workload can safely absorb the lower frequency state. Furthermore, to resolve severe bottlenecks where DVFS is rendered unsafe, the framework embeds a contention-aware task migration heuristic that leverages the exact same feature vector to balance system activity.

The following subsections detail the core components of this architecture-agnostic framework and their specific mathematical adaptations for global counting mechanism not originally covered in [6].

A. Workload Categorization

To rigorously profile the architectural bottlenecks across distinct contention profiles, the tailored real-time taskset Γ is composed of three heterogeneous classes of applications. These classes are designed to isolate specific microarchitectural stress points and validate the model's ability to discern varying degrees of shared resource dependency:

- **CPU-Bound (Compute-Intensive):** Tasks characterized by high instruction throughput and a negligible LLC footprint. We employ an iterative Fibonacci sequence generator utilizing dense floating-point operations. This class maintains high internal pipeline utilization and is highly resilient to external memory interference.
- **Memory-Bound (Bandwidth-Intensive):** Tasks explicitly designed to saturate the memory subsystem. These applications linearly traverse large arrays exceeding the capacity of the private L1 caches, actively thrashing the shared LLC and generating maximal inter-core interference $I(t)$.
- **Mid-Term (Balanced Vision Pipeline):** A hybrid workload representative of modern cyber-physical systems, specifically modeled after real-time multimedia streaming and autonomous vehicle perception pipelines. We utilize an image disparity calculation algorithm that exhibits phased execution: it alternates between heavy memory bandwidth utilization (fetching stereo image matrices every iteration) and intense CPU-bound arithmetic (calculating pixel disparities). This class dynamically shifts its bottleneck, challenging the ANN to adapt to non-linear contention patterns.

Table I presents the baseline task-set used to profile the architectural bottleneck. It comprises three distinct behaviors for the Image Disparity task: i) the Image disparity running in CPU 1, runs right after the memory-bound task, suffering from L1 cache contention; ii) the Image Disparity running on CPU 2 runs in parallel to the memory-bound task, suffering from L2 cache contention; iii) the Image Disparity running on CPU 3 is not expected to suffer from memory hierarchy contention. By profiling the architecture through this task perspective we are able to identify the main effects of shared resource contention into a mid-term task, while also accounting for baseline CPU-bound performance metrics.

B. Workload Architecture Adaptation

To validate the portability of the predictive framework from the baseline architecture in [**?**] to a novel platform,

TABLE I
PROFILING TASK-SET CONFIGURATION

Set	Core	Task Type	Period/Deadline (μs)	Budget (μs)	Learning
TS1	1	Bandwidth (Heavy)	1000	200	*Warm-Start*
	1	Image Disparity	1000	200	
	2	Image Disparity	1000	200	
	2	CPU-Hungry	1000	200	
	3	CPU-Hungry	1000	200	
	3	Image Disparity	1000	200	

the taskset Γ cannot be statically ported. Instead, it must be mathematically scaled to match the computational capacity and memory hierarchy of the target SoC.

Let the target architecture have a shared LLC capacity of S_{LLC} and an average Instructions Per Cycle (IPC) scaling factor of α relative to the baseline architecture. The taskset adaptation process ensures that the contention dynamics remain proportional by applying two transformations:

1) **Temporal Scaling:** The baseline isolated execution time C_i is adjusted by α to account for the new pipeline depth and maximum frequency. The period T_i is subsequently scaled to maintain the original per-core utilization factor $U = \sum(C_i/T_i)$, ensuring the PEDF schedulability bounds remain identical.
2) **Spatial Scaling:** To properly evaluate the contention-aware ML predictor, the taskset must actively induce measurable interference $I(t)$. If S_{LLC} of the new target architecture is larger than the baseline, the Working Set Size W_i of memory-bound and mid-term tasks is scaled such that $\sum_{i \in v_k} W_i > S_{LLC}$. This guarantees that the tasks exhaust the private L1 caches and actively contend for the shared LLC, recreating the necessary environmental stress to trigger the migration heuristics.

By adapting Γ to the architectural parameters of the target platform, we ensure that the ML framework is evaluated against functionally equivalent contention scenarios, preserving the integrity of the portability validation.

C. Profiling and Feature Selection

To systematically isolate the most deterministic indicators of temporal degradation without relying on human intuition, the framework employs a tripartite feature selection pipeline consisting of statistical, entropy-based, and embedded techniques. First, Pearson's Correlation Coefficient (PCC) is applied as a baseline filter to quantify the linear relationship between architectural events and execution time delays, while crucially mapping the covariance between the features themselves. Second, the Information Gain Ratio (IGR) evaluates the entropy reduction provided by each feature, highlighting non-linear dependencies by measuring how much uncertainty in the execution time is resolved by a specific counter's outcome. Finally, Lasso Cross-Validation (Lasso CV) acts as an embedded regression method utilizing L1 regularization. By heavily penalizing the absolute magnitude of regression coefficients, Lasso CV actively shrinks mathematically irrelevant or highly collinear features to zero, yielding an optimized subset tailored for predictive accuracy.

Because each algorithm utilizes a fundamentally distinct relevance metric, identifying the optimal feature-set cannot be achieved through a simple mathematical intersection of their top results. Instead, the framework merges the outputs through a consensus and redundancy-elimination strategy. Features that achieve high importance rankings across multiple methods form an initial candidate pool. To refine this pool and prevent multicollinearity from destabilizing the subsequent ANN training, the covariance matrix generated by PCC is cross-referenced against the candidates. When multiple top-ranking counters exhibit exceptionally high cross-correlation (e.g., exceeding 90%), only a single representative feature from that cluster is retained. This merging methodology safely distills thousands of raw platform events into a compact, information-dense vector, capturing orthogonal dynamics like memory bus saturation, cache thrashing, and pipeline stalls, ensuring the ML model is accurately primed to guide real-time DVFS actuation.

D. Global Events Adaptation

The transition of predictive scheduling models across diverse hardware ecosystems frequently encounters the challenge of opaque monitoring topologies at higher levels of the memory hierarchy. In some COTS multiprocessor systems, the hardware Performance Monitoring Units (PMUs) associated with shared resources, such as the LLC or main memory controllers, track performance events globally. For instance, the RISC-V platform StarFive JH7110 SoC, built around a quad-core SiFive U74 cluster, employs a unified 2MB L2 cache where performance events are strictly aggregated at the global level. These architectures fundamentally lack the microarchitectural support required to tag or partition cache misses and bus transactions by the originating hardware thread (hart).

Consequently, directly inputting a global contention metric (e.g., LLC_{global}) into a per-core ANN severely pollutes the feature vector. This aggregation prevents the machine learning model from isolating which specific core is actively generating the memory traffic and which is merely suffering from the resulting pipeline stalls. If left uncorrected, this ambiguity misleads the ANN, causing it to inaccurately predict diminishing returns or catastrophic deadline misses during CPU frequency scaling actuation.

To maintain the architecture-agnostic property of the framework without requiring custom silicon modifications, we introduce a proxy to disaggregate the global contention metric at the software level. We hypothesize that pipeline stalls induced by LLC contention are inversely proportional to the local instruction retirement rate (see Figure **??**). We utilize the fixed, per-core Instructions Retired (Instructions Retired (IR)) counter—mandated by the RISC-V privileged specification—as a normalizing denominator. For a given core i during a sampling period t, the Contention Penalty (CP_i) is calculated as:

$$CP_i = \frac{L2_{global}}{IR_i} \tag{1}$$

This ratio provides a dynamic, hardware-aware diminishing factor based on workload characteristics:

- **CPU-Centric Execution:** If core i executes a compute-bound task, it predominantly hits the private L1 cache, resulting in a high IR_i. The large denominator diminishes the impact of $L2_{global}$, correctly signaling to the ANN that the task is largely isolated from current shared memory contention.
- **Memory-Bound Execution:** If core i experiences LLC contention, its pipeline stalls, causing a severe drop in IR_i. The reduced denominator amplifies the CP_i value, allowing the ANN to accurately correlate the global L2 spike with the specific degradation of the task on core i.

By replacing the direct LLC miss count with the CP_i ratio, the input layer of the ANN is successfully adapted to opaque U74-based architectures. This preprocessing step adds exactly one floating-point division per core to the monitoring overhead, preserving the strict temporal constraints required for real-time execution time prediction.

E. Online Execution Time Prediction

The predictive core of the architecture-agnostic framework is driven by a lightweight, feed-forward ANN integrated directly into the real-time scheduler. Rather than relying on static, highly pessimistic worst-case execution time bounds, the ANN dynamically estimates the execution time (C_{pred}) of a task under a proposed lower frequency state.

During the execution of a job, the modified performance monitoring layer collects the platform's feature vector, which includes the current operating frequency and the newly adapted contention proxy metrics (e.g., CP_i). Upon job completion, the actual execution time (C_{actual}) is measured and compared against the previously predicted value. The resulting error margin triggers an online backpropagation algorithm that continuously updates the ANN synaptic weights during runtime.

This continuous online learning paradigm is the critical enabler for cross-ISA portability: it completely eliminates the need for exhaustive, platform-specific offline profiling. As the taskset executes on the VisionFive 2, the ANN autonomously learns to map the complex, non-linear relationships between the specific RISC-V pipeline stalls, the scaled clock speeds, and the resulting temporal delays, adapting to the target microarchitecture organically.

F. Contention-Aware DVFS and Task Migration

While DVFS is highly effective for saving energy during compute-bound execution, blindly scaling down the frequency of a cluster saturated by memory-bound interference often leads to deadline violations. When the ANN predicts that scaling to a lower frequency f will induce a temporal failure ($C_{pred} > D_i$), the framework temporarily vetoes the DVFS actuation to preserve hard real-time schedulability.

However, simply locking the cluster at maximum frequency does not resolve the underlying architectural bottleneck. To actively mitigate severe inter-core interference, the framework employs a contention-aware migration heuristic. In the 2022 baseline architecture, this heuristic ranked tasks using raw, per-core LLC misses. Adapted for the opaque RISC-V platform, the scheduler now evaluates a weighted activity vector based on the mathematical proxy CP_i.

When a thermal or resource bottleneck is detected, the scheduler ranks the co-located tasks and identifies the application exhibiting the highest memory-bound interference footprint. This aggressive task is subsequently migrated to an alternate frequency cluster $\mathcal{V}$ with available memory bandwidth. By spatially redistributing the workload, the local LLC pressure is immediately alleviated. This dynamic resolution of shared resource contention ultimately restores the execution slack required for the ANN to safely resume energy-saving DVFS actuations.

V. Experimental Evaluation

To empirically validate the architecture-agnostic property of the proposed predictive framework and its mathematical adaptation for opaque performance monitoring, we deployed the system on a modern COTS RISC-V development platform.

A. Experimental Setup

All evaluations were conducted on the StarFive VisionFive 2 Single Board Computer. This platform is powered by the JH7110 SoC, which features a quad-core SiFive U74 (RV64GC) 64-bit RISC-V processor cluster. The memory hierarchy includes 32KB private L1 instruction and data caches per core, backed by a unified, shared 2MB LLC (L2 cache). The platform supports DVFS at the cluster level, providing a set of discrete operating frequencies ranging from 375 MHz to 1.5 GHz. The hardware PMU handles L2 cache events globally, requiring our software-level proxy to disaggregate the contention signals.

Real-Time Operating System: To guarantee deterministic execution and precisely measure microarchitectural overheads without the scheduling noise inherent to general-purpose operating systems, the framework was implemented natively within the Embedded Parallel Operating System (EPOS) [**?**]. EPOS is a component-based RTOS designed to deliver strict timing guarantees for cyber-physical applications. The ANN, the contention-aware migration heuristic, and the Partitioned EDF (PEDF) scheduler were deeply integrated into the EPOS kernel. To interface with the RISC-V hardware, EPOS accesses the local IR counters directly, while global L2 queries are executed via standard Supervisor Binary Interface (SBI) calls to the machine-mode firmware (OpenSBI).

To support the continuous data acquisition required by the predictive model, the framework relies on a non-intrusive monitoring abstraction layer, originally introduced in [8].

This mechanism provides a unified interface to sample hardware transducers, PMU event counters, and OS scheduling statistics without distorting the underlying system behavior. Because active polling would introduce memory traffic and pollute the shared LLC—thereby invalidating the contention measurements—the monitoring engine is explicitly designed for minimal interference. It achieves this by piggybacking all sampling operations onto pre-existing kernel routines, such as thread context switches. By leveraging optimized data structures during these mandatory OS interruptions, the framework guarantees negligible temporal overhead and prevents any scheduling jitter that could compromise the taskset's real-time integrity.

Workload Configuration and Scaling: To rigorously stress the ANN and force the migration heuristic to actuate, the abstract task classes defined in Section III were instantiated and scaled to exceed the architectural limits of the JH7110 SoC.

- *CPU-Bound:* The iterative Fibonacci generator was configured to sustain deep pipeline saturation using double-precision floating-point arithmetic.
- *Memory-Bound:* The bandwidth-intensive array traversal tasks were scaled to utilize a Working Set Size (WSS) of 1.5MB per task. When co-scheduled on the quad-core cluster, the aggregate memory footprint severely oversubscribes the 2MB shared LLC, deliberately inducing catastrophic inter-core interference and continuous pipeline stalls.
- *Mid-Term (Vision Pipeline):* The image disparity algorithm was fed with high-resolution stereo image matrices. This real-world proxy forces rapid, non-linear shifts between L1 cache hits (during pixel block matching) and L2/main memory fetches (during frame loads), continuously challenging the ANN's predictive accuracy under a dynamic DVFS regime.

B. Task-Set Configurations and Training Methodology

To rigorously evaluate the predictive framework's ability to map contention on the novel RISC-V architecture, we synthesized three distinct multiprogrammed task-sets from the workload classes defined previously. The tasks were statically partitioned across cores 1, 2, and 3 of the SiFive U74 cluster, leaving core 0 reserved for the EPOS RTOS tick and platform-level ANN prediction and voting.

The evaluation strategy isolates the initial architecture profiling from continuous run-time adaptation. The configurations are defined as follows:

- **Task-Set 1 (Homogeneous Warm-Start):** This configuration consists of six tasks with uniform periods and implicit deadlines ($T = D = 1,000\mu s$), evenly distributed across the three available application classes (Bandwidth, Disparity, CPU-Hungry). This symmetric alignment forces highly predictable, persistent contention phases. Task-Set 1 was executed in an offline profiling phase to *warm-start* the ANN. This provided the model with an initial baseline weight configuration corresponding to the maximum LLC contention penalty on the new JH7110 SoC.
- **Task-Set 2 (Heterogeneous Temporal Adaptation):** Moving away from uniform execution, this configuration introduces severe variance in task periods ($T \in [125, 1000\mu s]$) and execution budgets. The differing periods cause the execution phases of memory-bound and compute-bound tasks to interleave chaotically, creating dynamic, transient contention spikes. Crucially, the ANN was subjected to this task-set relying *completely on online learning*. The model had to adjust its weights dynamically during execution to predict these newly introduced interference patterns without any prior offline profiling for this specific temporal arrangement.
- **Task-Set 3 (Asymmetric Contention Stress):** To stress the contention-aware migration heuristic, the final configuration increases the multiprogramming level to seven tasks and introduces asymmetric memory demands (e.g., combining heavy image disparity tasks with lightweight, high-frequency bandwidth thrashers). Core 3 is heavily oversubscribed with diverse workloads. Similar to Task-Set 2, the ANN adaptation for this highly unpredictable workload was handled entirely online, validating the framework's robustness and architectural agility.

TABLE II
TASK-SET CONFIGURATIONS ON THE SIFIVE U74 CLUSTER

Set	Core	Task Type	Period/Deadline (μs)	Budget (μs)	Learning
TS2	1	Bandwidth (Heavy)	250	50	*Online*
	1	Image Disparity	500	100	
	2	Image Disparity	1000	200	
	2	CPU-Hungry	250	35	
	3	CPU-Hungry	125	20	
	3	Image Disparity	250	100	
TS3	1	Bandwidth (Light)	100	10	*Online*
	2	Bandwidth (Light)	100	5	
	2	Image Disparity	1000	400	
	3	CPU-Hungry	100	30	
	3	Image Disparity	500	100	
	3	CPU-Hungry	250	60	
	3	Bandwidth (Light)	1000	100	

C. Prediction Accuracy and Convergence

D. Run-Time Overhead Analysis

E. Energy Optimization

F. Schedulability Assurance

VI. Conclusion

VII. Data Analysis

Acknowledgments

This work was partially funded by Fundação de Apoio da UFMG (Fundep), through Linha VI – Conectividade Veicular, a prioritary program from Mover (Mobilidade Verde e Inovação), project AutoDL (29271.03.01/2023.04-00).

REFERENCES

[1] S. Rahim, P. P. Nair, and R. Devaraj, "A survey of machine learning-driven task scheduling approaches for multiprocessor systems," *Journal of Systems Architecture*, p. 103628, 2025.

[2] G. Gracioli, R. Tabish, R. Mancuso, R. Mirosanlou, R. Pellizzoni, and M. Caccamo, "Designing mixed criticality applications on modern heterogeneous mpsoc platforms," in *31st Euromicro Conference on Real-Time Systems (ECRTS 2019)*. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2019, pp. 27–1.

[3] L. P. Horstmann, J. L. C. Hoffmann, and A. A. Fröhlich, "A framework to design and implement real-time multicore schedulers using machine learning," in *2019 24th IEEE international conference on emerging technologies and factory automation (ETFA)*. IEEE, 2019, pp. 251–258.

[4] T. Lugo, S. Lozano, J. Fernández, and J. Carretero, "A survey of techniques for reducing interference in real-time applications on multicore platforms," *IEEE Access*, vol. 10, pp. 21 853–21 882, 2022.

[5] S. Das, J. Werner, M. Antonakakis, M. Polychronakis, and F. Monrose, "Sok: The challenges, pitfalls, and perils of using hardware performance counters for security," in *2019 IEEE symposium on security and privacy (SP)*. IEEE, 2019, pp. 20–38.

[6] J. L. C. Hoffmann and A. A. Fröhlich, "Online machine learning for energy-aware multicore real-time embedded systems," *IEEE Transactions on Computers*, vol. 71, no. 2, pp. 493–505, 2021.

[7] J. L. C. Hoffmann, J. L. De Souza, and A. A. Fröhlich, "Lightweight and robust online learning for energy-aware multicore real-time schedulers," in *2024 XIV Brazilian Symposium on Computing Systems Engineering (SBESC)*. IEEE, 2024, pp. 1–6.

[8] L. P. Horstmann, J. L. C. Hoffmann, and A. A. Frohlich, "A framework to design and implement real-time multicore schedulers using machine learning," in *2019 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, Sep. 2019.